

راهنمای گام به گام طراحی و تصمیم‌گیری در معماری نرم افزار

معماری نرم افزار یک فرایند چندمرحله‌ای است که در هر مرحله نیاز به تصمیم‌گیری‌های مهم و مستندسازی دقیق دارد. در ادامه، یک **فلوچارت تصمیم‌گیری گام به گام** ارائه می‌شود که به معماران ارشد، رهبران فنی و توسعه‌دهندگان مسئول طراحی کمک می‌کند تا مسیر طراحی را به صورت ساخت‌یافته پیش ببرند. این راهنما بر اساس مفاهیم کتاب *Fundamentals of Software Architecture: An Engineering Approach* تدوین شده و با منابع معتبر دیگر تکمیل شده است.

مرحله اول: تحلیل پایه‌ها و تعریف نیازمندی‌ها (Requirements Analysis & Foundations)

در ابتدای کار، باید مبانی پروژه و نیازمندی‌های کلان را کاملاً درک کنید. هدف این مرحله فهم دقیق مسئله، محدوده‌ی کسب‌وکار، و معیارهای موفقیت سیستم است.

۱. درک نقش معمار و انتظارات آن

نخست مشخص کنید در نقش معمار چه مسئولیت‌ها و انتظاراتی بر عهده دارید. یک معمار نرم افزار موفق باید وظایف زیر را انجام دهد ¹:

- **اتخاذ تصمیمات معماری:** تصمیم‌گیری در مورد ساختار و راهبردهای فنی کلان (نه لزوماً جزئیات فناوری) که مسیر پروژه را هدایت کند ¹. این تصمیمات شامل انتخاب سبک معماری، الگوهای اصلی، و قوانین حاکم بر سیستم است.
- **تحلیل مداوم معماری:** به صورت مستمر ارزیابی کنید که معماری کنونی تا چه حد پاسخگوی نیازها است و آیا با گذشت زمان نیاز به تغییر دارد ¹. به خصوص اگر معماری موجود قدیمی‌تر از چند سال باشد، ممکن است نیاز به بازبینی داشته باشد.
- **به روز نگه داشتن دانش فنی:** روندهای جدید فناوری و معماری را پیگیری کنید ¹. قانون معروف "روزانه ۲۰ دقیقه برای یادگیری" را رعایت کنید؛ مثلاً هر روز صبح پیش از شروع کار حدود ۲۰ دقیقه صرف مطالعه فناوری‌ها یا الگوهای نوین کنید ². این کار به شما کمک می‌کند همواره با دانش روز تصمیم‌گیری نمایید.
- **نظارت بر پایبندی به تصمیمات:** اطمینان حاصل کنید که تیم توسعه طبق تصمیمات و اصول معماری تعیین‌شده عمل می‌کند ¹. این نظارت می‌تواند از طریق کدبازبینی، گفتگوهای منظم با تیم و تعریف معیارهای سنجش انجام شود.

○ **نکته:** نقش معمار فراتر از تصمیم‌گیری فنی صرف است. معمار باید **دید کسب‌وکاری** داشته باشد و مهارت‌های ارتباطی بالایی به خرج دهد تا تصمیمات خود را برای ذی‌نفعان مختلف توجیه کند ³. به خاطر داشته باشید که تقریباً تمام تصمیمات معماری با چالش و پرسش مواجه خواهند شد ³، بنابراین توانایی استدلال و اقناع از ویژگی‌های کلیدی شماست.

۲. شناسایی ویژگی‌های معماری (Architectural Characteristics)

ویژگی‌های معماری (یا همان صفات کیفی سیستم که گاهی "ilities" نیز نامیده می‌شوند) معیارهای موفقیت و اولویت‌های غیروظیفه‌ای سیستم را تشکیل می‌دهند ⁴. این ویژگی‌ها مستقل از قابلیت‌های عملکردی هستند و

مواردی مانند امنیت، عملکرد، مقیاس‌پذیری، دسترس‌پذیری، قابلیت نگهداری و غیره را شامل می‌شوند⁴. اقدامات زیر را انجام دهید:

- **استخراج ویژگی‌ها از نیازمندی‌ها:** نیازمندی‌های کسب‌وکار و دغدغه‌های ذی‌نفعان را مرور کنید و ویژگی‌های کیفی مهم را استخراج نمایید. برخی ویژگی‌ها به صورت **صریح** در اسناد نیازمندی ذکر شده‌اند؛ برای مثال ممکن است در مستندات به "زمان پاسخ زیر 2 ثانیه" اشاره شود که بیانگر ویژگی «کارایی» است. در کنار این‌ها، ویژگی‌های **ضمنی** را نیز در نظر بگیرید - حتی اگر در مستندات نیامده باشد، معمولاً هر سیستمی به امنیت، **قابلیت اطمینان** و **دسترس‌پذیری** نیاز دارد⁴. این‌ها را به عنوان ویژگی‌های ضمنی شناسایی و ثبت کنید.
- **مرتبط ساختن ویژگی‌ها با اهداف کسب‌وکار:** هر ویژگی معماری باید بازتابی از یک دغدغه یا هدف تجاری باشد. به عنوان نمونه، اگر هدف کسب‌وکار «کاهش زمان عرضه محصول به بازار» است، ویژگی‌های معماری مرتبط می‌توانند «چابکی (Agility)»، «تست‌پذیری» و «استقرارپذیری» باشند⁵. این ارتباط را مستند کنید تا بعدها بتوانید توجیه کنید چرا یک ویژگی مهم است.
- **اولویت‌دهی و محدود کردن تعداد ویژگی‌ها:** یکی از دام‌های معماری این است که سعی کنیم سیستم را از همه جهات عالی کنیم - در حالی که این ممکن نیست. بنابراین از ذی‌نفعان بخواهید **حداکثر سه ویژگی کیفی کلیدی** را که برای موفقیت پروژه حیاتی‌اند انتخاب کنند. پشتیبانی از تعداد زیادی ویژگی معماری باعث پیچیدگی و تضاد در تصمیمات می‌شود⁴. با محدود کردن تمرکز به مهم‌ترین‌ها، طراحی ساده‌تر و منسجم‌تری خواهید داشت.
- **مستندسازی تعریف و معیار هر ویژگی:** برای هر ویژگی منتخب، تعریف دقیقی بنویسید و مشخص کنید چگونه قابل اندازه‌گیری یا آزمون است (مثلاً زمان پاسخ باید کمتر از X ثانیه باشد یا سیستم باید N کاربر همزمان را پشتیبانی کند). این کار در مراحل بعدی برای طراحی آزمون‌های تناسب (Fitness Functions) مفید خواهد بود.

۳. تحلیل مبادله‌ها (Trade-off Analysis)

- مهم‌ترین قانون معماری نرم‌افزار این است که «**همه چیز نوعی مبادله (Trade-off) است**»⁶. هر تصمیمی که به نفع یک ویژگی معماری می‌گیرید ممکن است به قیمت تضعیف ویژگی دیگر تمام شود. در این گام:
- **تعارضات بین ویژگی‌ها را شناسایی کنید:** برای هر یک از ویژگی‌های معماری اولویت‌دار، بررسی کنید که تقویت آن چه پیامدهایی برای سایر ویژگی‌ها دارد. به عنوان مثال، ممکن است افزایش امنیت منجر به کاهش اندکی در **کارایی** یا **مقیاس‌پذیری** شود. یا ارتقای **سهولت تست‌پذیری** ممکن است پیچیدگی پیاده‌سازی را کمی افزایش دهد. این مبادله‌ها را یادداشت و برای خودتان شفاف کنید.
 - **اولویت‌بندی در صورت تعارض:** در صورت تعارض مستقیم بین دو ویژگی (مثلاً امنیت در برابر مقیاس‌پذیری)، تصمیم بگیرید کدامیک در اولویت بالاتری قرار می‌گیرد⁶. این تصمیم باید بر اساس نیاز کسب‌وکار و خواست ذی‌نفعان باشد. برای نمونه، اگر داده‌های بسیار حساس را نگهداری می‌کنید، شاید امنیت ارزشمندتر از چند میلی‌ثانیه بهبود کارایی باشد.
 - **ثبت منطق مبادله:** دلایل این ترجیح را مستند کنید (برای مثال: "امنیت اولویت بالاتری دارد زیرا نشت داده به اعتبار شرکت آسیب جدی خواهد زد"⁶). بعدها در هنگام مستندسازی تصمیمات معماری، این دلایل به کار می‌آیند و به ذی‌نفعان نشان می‌دهند که تصمیمات به صورت سنجیده و آگاهانه اتخاذ شده است.

اصل راهنما: معماری "بهترین" وجود ندارد؛ شما در واقع به دنبال کم‌هزینه‌ترین معماری نامناسب هستید که نیازها را برآورده کند⁷ ⁸ ! این یعنی با پذیرش این که هر گزینه‌ای نقاط ضعفی دارد، گزینه‌ای را انتخاب کنید که نقاط ضعف آن در زمینه‌ی نیازهای شما کمترین تأثیر منفی را داشته باشد.

۴. تعیین محدوده‌ی کوانتوم معماری (Architectural Quantum)

- کوانتوم معماری مفهوم مهمی است که محدوده و یکپارچگی اجزای مستقل سیستم را تعریف می‌کند⁹. هر **کوانتوم معماری** به زبان ساده یعنی یک جزء یا زیرسیستم مستقل که می‌تواند جداگانه استقرار یابد و ویژگی‌های معماری خود را به طور کامل در بر دارد⁹. در این مرحله:
- **تعریف کوانتوم معماری:** یک کوانتوم معماری عبارت است از یک جزء قابل استقرار مستقل که انسجام عملکردی بالایی دارد و اجزایش به صورت هم‌زمان به هم وابسته‌اند (Connascence هم‌زمان)⁹. به بیان دیگر، همه اجزای داخل

یک کوانتوم برای کار صحیح نیاز دارند با نسخه مشخصی از یکدیگر همراه باشند و اغلب پایگاه داده مختص خود را نیز شامل می‌شوند⁹.

- **تعیین تعداد کوانتوم‌های سیستم:** با توجه به ویژگی‌های معماری مرحله قبل، ببینید آیا همه بخش‌های سیستم به یک سطح کیفی یکسان نیاز دارند یا خیر. اگر **تمام** بخش‌های سیستم قرار است همان سه ویژگی معماری برتر را داشته باشند (برای مثال همه اجزا باید بسیار در دسترس و بسیار مقیاس‌پذیر باشند)، آنگاه شاید کل سیستم را بتوان یک کوانتوم بزرگ در نظر گرفت. این حالت معمولاً منجر به یک معماری یکپارچه (Monolithic) می‌شود. اما اگر بخش‌های مختلف نیازمندی‌های کیفی متفاوتی دارند یا استقلال استقراری مهم است - مثلاً بخش پرداخت نیاز به امنیت و قابلیت اطمینان فوق‌العاده دارد اما سایر بخش‌ها این‌طور نیستند - در این صورت سیستم شما ممکن است نیاز به چندین کوانتوم داشته باشد⁹. هر کوانتوم می‌تواند به عنوان یک سرویس یا ریزسرویس جداگانه طراحی و مستقر شود.

- **تصمیم‌گیری اولیه: معماری یکپارچه یا توزیع‌شده؟** خروجی تحلیل کوانتوم یک راهنمای کلان به شما می‌دهد: اگر فقط یک کوانتوم بزرگ دارید، احتمالاً یک معماری **یکپارچه** مناسب‌تر است؛ ولی اگر چند کوانتوم شناسایی کردید، به سوی معماری **توزیع‌شده** (مانند مایکروسرویس‌ها یا سرویس‌گرا) حرکت کنید⁹. توجه داشته باشید که این تصمیم اولیه است و در مرحله طراحی ساختار کلان دقیق‌تر بررسی خواهد شد.

چک‌لیست مرحله ۱ - تصمیمات و اقدامات:

- [] تعریف کرده‌ام که نقش من به عنوان معمار شامل چه مسئولیت‌هایی است و انتظارات تیم/سازمان چیست.

- [] ویژگی‌های کیفی (معماری) مهم سیستم را از دل نیازمندی‌ها و اهداف کسب‌وکار استخراج و **سه مورد اولویت‌دار** را انتخاب کرده‌ام.

- [] برای هر ویژگی معماری، مبادله‌های بالقوه با سایر ویژگی‌ها را تحلیل و در صورت تعارض، اولویت‌بندی واضحی تعیین کرده‌ام.

- [] مشخص کرده‌ام سیستم به چند **کوانتوم معماری** نیاز دارد و در نتیجه تمایل معماری کلان (یکپارچه در برابر توزیع‌شده) را معلوم کرده‌ام.

</hr>

مرحله دوم: طراحی ساختار کلان (Macro-Architecture Design)

پس از تعیین نیازمندی‌های کیفی و محدوده‌های کوانتومی، نوبت به طراحی ساختار اصلی سیستم می‌رسد. در این مرحله سبک معماری مناسب، مؤلفه‌های درگیر و نحوه تعامل آن‌ها را مشخص می‌کنید.

۱. شناسایی مؤلفه‌های اصلی سیستم

مؤلفه ها (Components) بلوک‌های فیزیکی اصلی در معماری هستند که وظایف یا زیرسیستم‌های منطقی را نمایندگی می‌کنند¹⁰. شناسایی و تفکیک مناسب مؤلفه‌ها، پایه‌ی یک طراحی ماژولار و قابل فهم را بنا می‌نهد. برای تعیین مؤلفه‌های اولیه این اقدامات را انجام دهید:

- **انتخاب رویکرد تفکیک اولیه:** تصمیم بگیرید تقسیم‌بندی کلان سیستم در ابتدا به صورت **فنی/لایه‌ای** باشد یا **دامنه‌محور**. در رویکرد **لایه‌ای** (Technical Partitioning)، مؤلفه‌ها بر اساس نقش‌های فنی مثل لایه نمایش، لایه منطق کسب‌وکار و لایه داده تفکیک می‌شوند¹¹. در رویکرد **دامنه‌محور** (Domain Partitioning)، هر مؤلفه حول یک قابلیت دامنه یا زیردامنه‌ی کسب‌وکاری شکل می‌گیرد - مانند مؤلفه‌های «پرداخت»، «سفرارش‌ها» یا «کاتالوگ محصولات»¹¹. به عنوان مثال، اگر از معماری **مدولار یکپارچه** یا **میکروسرویس‌ها** استفاده کنید، معمولاً رویکرد دامنه‌محور را برمی‌گزینید، در حالی که در معماری **لایه‌ای کلاسیک** از رویکرد فنی استفاده می‌شود¹¹.

- **روش‌های کشف مؤلفه‌ها:** از تکنیک‌های تکرارشونده برای شناسایی مرزهای مؤلفه‌ها بهره ببرید. یکی از روش‌های موثر **مدل بازیگر-کنش (Actor/Action)** است که با ترسیم نقش‌ها (کاربران یا سیستم‌های خارجی) و موارد استفاده، به استخراج مؤلفه‌های کاندید کمک می‌کند¹². روش دیگر **طوفان رویداد (Event Storming)** است که رویدادهای کسب‌وکار و جریان‌های کاری را مدل می‌کند و از دل آن‌ها سرویس‌ها/مؤلفه‌ها را می‌یابد¹². همچنین

می‌توانید از مدل‌سازی گردش کار استفاده کنید و گام‌های فرآیندهای اصلی را به اجزای مسئول هر گام نسبت دهید.¹²

- **اجتناب از ضدالگوی "مدیر موجودیت"**: مراقب باشید که معماری را صرفاً بر اساس موجودیت‌های پایگاه‌داده تعریف نکنید. ضدالگوی رایجی وجود دارد که به ازای هر جدول یا موجودیت اصلی، یک مؤلفه‌ی نرم‌افزاری مثلاً `CustomerManager` یا `OrderManager` تعریف می‌شود - بدون آن‌که ملاحظات رفتاری/کاربردی در نظر گرفته شود.¹³ این رویکرد معمولاً به ماژول‌هایی با انسجام پایین و وابستگی زیاد منجر می‌شود. به جای آن، تمرکز را بر **مرزبندی مبتنی بر رفتارهای دامنه و موارد استفاده** بگذارید نه صرفاً موجودیت‌های داده.

نمونه‌ای از فلوچارت فرآیند شناسایی مؤلفه‌ها از کتاب *Richards & Ford* - این نمودار مراحل تعیین و بازبینی مؤلفه‌های اولیه را نشان می‌دهد (تعریف مؤلفه‌ها، اختصاص نیازمندی‌ها به آن‌ها، بررسی مسئولیت‌ها و ویژگی‌های کیفی، و در صورت نیاز بازنگری و تعدیل مرزبندی مؤلفه‌ها). با استفاده از این چرخه می‌توانید به تدریج مؤلفه‌های پایدار و با انسجام بالا را شناسایی کنید.

۲. انتخاب سبک (الگوی) معماری مناسب

در این مرحله، بر اساس کوانتوم‌های شناسایی‌شده و نیازمندی‌های عملکردی و کیفی، الگوی معماری اصلی سیستم را تعیین کنید. پرسش اساسی این است: **معماری یکپارچه یا توزیع‌شده؟** و همچنین دقیق‌تر: کدام سبک مشخص (لایه‌ای، میکروسرویسی، رویدادمحور و غیره) برای مسئله‌ی ما مناسب‌تر است. تصمیم‌گیری درست در اینجا تأثیر زیادی بر موفقیت پروژه دارد.¹⁴

- **یکپارچه در برابر توزیع‌شده**: همان‌طور که گفته شد تحلیل کوانتوم یک راهنمای اولیه است. اگر سیستم شما یک کوانتوم بزرگ دارد (همه اجزا به هم پیوسته‌اند و نیازهای کیفی یکسانی دارند)، معماری **Monolithic (یکپارچه)** ساده‌تر و کم‌هزینه‌تر خواهد بود. اما اگر چند کوانتوم مستقل دارید (زیرسیستم‌های نسبتاً خودمختار با پایگاه داده‌ها یا نیازهای متفاوت)، معماری **Distributed (توزیع‌شده)** گزینه مناسب‌تری است که اجازه می‌دهد هر بخش را جداگانه توسعه و استقرار دهید.⁹

- **بررسی گزینه‌های رایج**: جدول زیر تعدادی از سبک‌های معماری شناخته‌شده را مقایسه می‌کند. با توجه به ویژگی‌های کیفی اولویت‌دار خود، می‌توانید مناسب‌ترین را انتخاب کنید.

سبک معماری	نوع استقرار	مناسب برای ویژگی‌های معماری (موارد برجسته)	مبادله‌ها و معایب کلیدی
معماری لایه‌ای (Layered)	یکپارچه	سادگی در توسعه، هزینه‌ی پایین برای شروع، ساختار قابل‌درک ¹⁵	مقیاس‌پذیری محدود و کاهش چابکی در گذر زمان (افزایش کوپلینگ بین لایه‌ها) ¹⁷
معماری میکروکرنل (Microkernel) (پلاگین)	یکپارچه	امکان سفارشی‌سازی و افزونه‌پذیری بالا (هسته پایدار + افزونه‌ها)	چالش در مقیاس‌پذیری بالا، افت قابلیت اطمینان در صورت خرابی هسته یا افزونه‌ها (پیچیدگی ارتباط بین آن‌ها)
معماری سرویس‌محور (Service-Based)	توزیع‌شده	چابکی متوسط، استقرارپذیری مستقل هر سرویس، قابلیت اطمینان بهبودیافته (در حد ACID چون معمولاً DB اشتراکی)	برخی محدودیت‌های مقیاس‌پذیری (به دلیل اشتراک منابع مثل پایگاه‌داده) و نیاز به هماهنگی بین سرویس‌ها

سبک معماری	نوع استقرار	مناسب برای ویژگی‌های معماری (موارد برجسته)	مبادله‌ها و معایب کلیدی
معماری میکروسرویس‌ها (Microservices)	توزیع شده	چابکی و توسعه مستقل بالا، تست‌پذیری و استقرارپذیری عالی ^{18 19}	پیچیدگی عملیاتی (چالش‌های شبکه‌ای، مدیریت سرویس‌های زیاد)، نیازمند تیم بالغ و استانداردهای بالا ^{20 21}
معماری رویدادمحور (EDA) (Event-Driven)	توزیع شده	عملکرد و مقیاس‌پذیری بالا، واکنش‌گرایی آنی به رویدادها ^{22 23}	دشواری در ردیابی جریان‌کار (در مدل‌های کاملاً توزیع شده)، پیچیدگی در مدیریت خطاها (به‌ویژه در توپولوژی Broker) ²³
معماری مبتنی بر فضا (Space-Based)	توزیع شده	مقیاس‌پذیری بسیار زیاد با افزودن گره‌ها، انعطاف‌پذیری (Elasticity) و کارایی بالا (همه چیز در حافظه) ^{24 25}	پیچیدگی بسیار بالا در پیاده‌سازی و پشتیبانی، دشواری تست جامع سیستم (به خاطر توزیع‌شدگی شدید داده و منطق)

• **مثال تصمیم‌گیری Monolith vs. Microservices:** تصمیم بین معماری یکپارچه یا ریزسرویس‌ها در بسیاری از پروژه‌ها چالشی اساسی است. **سایز تیم/سازمان و پیچیدگی دامنه** از عوامل تعیین‌کننده هستند²⁶. اگر تنها یک تیم کوچک روی سیستم کار می‌کند، یک معماری یکپارچه اغلب **ساده‌تر و کم‌هزینه‌تر** است. در مقابل، سازمانی با چندین تیم مستقل ممکن است با معماری مایکروسرویسی **چابک‌تر و مقیاس‌پذیرتر** باشد (مطابق قانون کانوی: ساختار سیستم شما بازتابی از ساختار سازمانی شماست²⁷). شکل زیر یک نمودار تصمیم‌گیری نمونه برای انتخاب بین مونولیت و میکروسرویس در یک پروژه جدید را نشان می‌دهد که برخی از این عوامل را در نظر می‌گیرد:

نمونه فلوجارت تصمیم‌گیری برای انتخاب معماری یکپارچه یا مایکروسرویسی (برگرفته از تجربه صنعتی). در این نمودار پرسش‌هایی درباره تعداد تیم‌های توسعه، نیاز به مقیاس افقی، سطح استقلال ماژول‌ها و پیچیدگی پروژه پرسیده می‌شود. مسیرهای تصمیمی مختلف ممکن است به انتخاب مونولیت سنتی، مونولیت ماژولار، سیستم توزیع‌شده سرویس‌گرا یا معماری مایکروسرویس کامل ختم شوند. همان‌طور که دیده می‌شود، اگر تیم کوچک و متمرکز باشد یا نیاز به اشتراک‌دهی و هماهنگی تنگاتنگ باشد، تمایل به معماری یکپارچه بیشتر است؛ در حالی که تیم‌های متعدد با حوزه‌های مستقل، معماری مایکروسرویسی را می‌طلبند. نتیجه نهایی باید با در نظرگیری غیرفنی‌ها (مانند ساختار سازمانی و هزینه‌ها) و فنی‌ها اتخاذ شود.^{26 28}

۳. تعیین سبک تعامل و ارتباطات بین مؤلفه‌ها

پس از انتخاب سبک کلی معماری و تعیین مؤلفه‌ها/سرویس‌ها، باید تصمیم بگیرید این اجزا چگونه با هم ارتباط برقرار کنند. معماری سیستم تنها شامل **چه اجزایی** نیست، بلکه **نحوه ارتباط آن‌ها** نیز بخش مهمی از طراحی معماری است. دو بعد کلیدی در این زمینه وجود دارد:

- **ارتباط همزمان (Synchronous) در برابر غیرهمزمان (Asynchronous):** به طور پیش‌فرض در معماری‌های داخلی (مانند یکپارچه) یا هر کجا سادگی مد نظر است، **ارتباط همزمان** (مانند فراخوانی مستقیم متدها یا API‌های REST که پاسخ فوری می‌دهند) انتخاب معقولی است، زیرا طراحی، پیاده‌سازی و اشکال‌زدایی آن ساده‌تر است²⁹. اما اگر سیستم نیازمند **واکنش‌پذیری بالا، تحمل تأخیر و بافر کردن ترافیک** باشد - مثلاً در پردازش رویدادهای حجیم یا سیستم‌های غیرهمزمان - می‌توان از **ارتباط غیرهمزمان** (مانند صف‌های پیام یا رویدادها) بهره برد. همزمانی ساده است ولی مقیاس‌پذیری محدودی دارد؛ غیرهمزمانی پیچیده‌تر است اما انعطاف و مقیاس‌پذیری بیشتری فراهم می‌کند. معیار تصمیم شما نیازمندی‌های غیرعملکردی (مثل زمان پاسخ قابل قبول، تحمل قطعی‌ها، و غیره) خواهد بود.
- **سبک هماهنگی بین سرویس‌های توزیع شده:** در سیستم‌های توزیع شده (مانند مایکروسرویس‌ها) دو الگوی

حاکم برای مدیریت جریان کار سرویس‌ها مطرح است: **ارکستریشن (Orchestration)** و **کورئوگرافی (Choreography)** ^{30 31} . در روش ارکستریشن، یک جزء مرکزی (مثل *Service Orchestrator* یا یک *Workflow Engine*) وجود دارد که ترتیب انجام کارها توسط سرویس‌های مختلف را دیکته می‌کند - شبیه رهبر ارکستر که به هر نوازنده می‌گوید چه زمانی بنوازد. این رویکرد **کنترل مرکزی** و دید جامعی روی جریان کار ایجاد می‌کند که برای فرآیندهای پیچیده مناسب است، اما نقطه‌ی تمرکز و وابستگی مرکزی نیز اضافه می‌کند. در مقابل، در روش کورئوگرافی سرویس‌ها بدون موجودیت مرکزی و با تبادل رویداد یا پیام، به صورت مستقل و **غیروابسته** تعامل می‌کنند - مانند رقص گروهی که هر کس بر اساس قواعد کلی و مشاهده دیگران حرکت خود را تنظیم می‌کند. این روش باعث **کاهش کوپلینگ** و افزایش استقلال سرویس‌ها می‌شود، اما ممکن است ردیابی توالی کل فرآیند و مدیریت خطا را سخت‌تر کند ^{32 23} . هنگام طراحی سیستم توزیع‌شده، بررسی کنید کدام سبک برای موارد استفاده شما مناسب‌تر است: برای **تراکنش‌های تجاری حساس با مراحل مشخص** ، ارکستریشن به دلیل کنترل صریح ترجیح دارد؛ اما برای **رویدادهای گسترده و انتشار-اشتراک** که نیاز به پاسخ‌گویی سریع و حداقل وابستگی دارند، کورئوگرافی گزینه بهتری است. (در عمل، بسیاری از معماری‌ها ترکیبی از هر دو رویکرد را در بخش‌های مختلف سیستم به کار می‌گیرند).

چکلیست مرحله ۲ - تصمیمات و اقدامات:

- [] مؤلفه‌ها/ماژول‌های اصلی سیستم را بر اساس لایه‌های فنی یا دامنه‌های کسب‌وکار شناسایی کرده‌ام و اطمینان دارم هر مؤلفه انسجام وظیفه‌ای مشخصی دارد.

- [] از تکنیک‌هایی مانند مدل‌سازی موارد استفاده، رویدادها یا گردش کار استفاده کرده‌ام تا مطمئن شوم هیچ مؤلفه‌ی مهمی را فراموش نکرده‌ام؛ همچنین از گرفتار شدن در دام «مدیر موجودیت» اجتناب کرده‌ام.

- [] سبک معماری کلان را انتخاب کرده‌ام (یکپارچه در مقابل توزیع‌شده، و نوع الگوی مشخص مانند لایه‌ای، میکروسرویسی، etc) و دلایل انتخابم را با توجه به ویژگی‌های کیفی و کوانتوم‌های معماری مستند کرده‌ام.

- [] نحوه‌ی ارتباط بین اجزا را طراحی کرده‌ام: همزمان یا غیرهمزمان بودن ارتباطات مشخص شده و در صورت معماری توزیع‌شده، رویکرد هماهنگی (ارکستریشن یا کورئوگرافی) هر بخش را تعیین نموده‌ام.

</hr>

مرحله سوم: تثبیت تصمیمات، حاکمیت و مدیریت ریسک (Stabilization, Governance & Risk Management)

پس از طراحی ساختار کلان، باید تصمیمات اتخاذشده را **مستند** و معیارهایی برای **کنترل و پایداری معماری** تعریف کنید. همچنین شناسایی و مدیریت ریسک‌های معماری در این مرحله بسیار حیاتی است.

۱. مستندسازی تصمیمات معماری در ADR

تصمیمات مهم معماری را در **سوابق تصمیم معماری** یا به اختصار (Architectural Decision Record) ADR ثبت کنید ³³ . هر ADR یک سند کوتاه است که یک تصمیم معماری و زمینه و دلایل آن را توضیح می‌دهد ^{34 35} . یک ADR استاندارد معمولاً شامل بخش‌های زیر است ³³ :

- **عنوان و وضعیت:** عنوانی توصیفی برای تصمیم (مثلاً "انتخاب پایگاه‌داده NoSQL برای ماژول X") به همراه وضعیت فعلی آن تصمیم. وضعیت می‌تواند پیشنهاد شده، تصویب شده یا اگر بعداً تصمیم دیگری جایگزینش شود کنار گذاشته/جایگزین شده باشد ³³ . شماره‌گذاری ADRها (مثلاً ADR-1, ADR-2, ...) نیز کمک می‌کند مرجع‌دهی به آن‌ها آسان‌تر شود.

- **زمینه (Context):** در این بخش توضیح می‌دهید چه شرایط و چالشی باعث شد این تصمیم مطرح شود ³⁶ . برای مثال: "ما نیاز به تحمل‌پذیری خطا در سطح دیتابیس داشتیم، به همین دلیل بررسی گزینه‌های پایگاه‌داده توزیع‌شده ضروری شد." این بخش کمک می‌کند خواننده بفهمد چه نیازی منجر به این تصمیم شده است.

- **تصمیم (Decision):** خود تصمیم را با جزئیات بیان کنید؛ یعنی چه انتخابی انجام دادید و چرا. دلایل کسب‌وکاری و فنی پشتیبان تصمیم را شرح دهید ³⁶ . مثلاً: "پایگاه‌داده Cassandra را انتخاب کردیم زیرا مدل داده توزیع‌شده و تحمل

خطای بالایی دارد که برای نیاز X مناسب است. با این انتخاب، در ازای پیچیدگی مدیریت خوشه، عملکرد خواندن سریعی به دست می‌آوریم." توجه کنید که چرا مهم‌تر از چه چیزی است - دیگران باید منطق پشت تصمیم را بفهمند ⁶ .

- **عواقب (Consequences):** پیامدهای تصمیم را صادقانه لیست کنید؛ شامل مزایا و معایب ³³ . هر تصمیمی trade-off دارد، بنابراین در این بخش توضیح دهید با این انتخاب چه چیزهایی به دست می‌آورد و چه هزینه‌ها یا خطراتی متحمل می‌شوید. همچنین اگر این تصمیم مستلزم اقدامات بعدی (مثلاً آموزش تیم، تغییر ابزارها، بازنگری در قسمت دیگری از سیستم) است ذکر کنید.

- **معیار پایبندی و نظارت (Compliance):** (اختیاری) اگر راهی برای نظارت خودکار یا دستی بر اجرای صحیح این تصمیم وجود دارد، توضیح دهید. برای نمونه اگر تصمیم گرفته‌اید که همه سرویس‌ها از OAuth2 برای احراز هویت استفاده کنند، در این بخش می‌توانید ذکر کنید که "بازبینی‌های کد و تست‌های امنیتی ماهانه انجام خواهد شد تا عدم انحراف از این تصمیم تضمین شود." این بخش بعدها به اجرای **توابع تناسب** مرتبط می‌شود.

ثبت ADRها مزایای زیادی دارد: همه افراد تیم از تصمیمات و دلایلشان مطلع می‌شوند، در آینده اگر افراد جدیدی به پروژه اضافه شوند می‌توانند منطق تصمیمات گذشته را دنبال کنند و در کل یک **حافظه معماری** برای پروژه ساخته می‌شود ³⁴ ³⁷ . به علاوه، هرگاه بخواهید تغییری در تصمیمی ایجاد کنید، با ایجاد ADR جدید و **Supersede** کردن تصمیم قبلی، سابقه تصمیم‌گیری سیستم را شفاف حفظ می‌کنید ³⁸ ³⁹ .

۲. تعریف توابع تناسب (Architecture Fitness Functions)

برای اطمینان از اینکه معماری طراحی‌شده در طول زمان و با تغییر کد دچار فرسایش نمی‌شود، از **توابع تناسب معماری** بهره بگیرید ⁴⁰ ⁴¹ . این ایده از معماری تکاملی می‌آید و به شما امکان می‌دهد مکانیزم‌هایی (اغلب خودکار) تعریف کنید که به طور مستمر وضعیت معماری را پایش کرده و در صورت انحراف، هشدار دهند یا جلوی آن را بگیرند ⁴⁰ .

- **تعریف:** تابع تناسب معماری هر مکانیزمی است که یک **سنجش عینی از یک یا چند ویژگی معماری** ارائه دهد ⁴¹ . به بیان ساده، توابع تناسب شبیه «تست‌های واحد برای معماری» هستند ⁴² ⁴³ . همان‌طور که تست‌های واحد صحت رفتارهای کد را می‌سنجند، توابع تناسب قواعد و قیود معماری را می‌سنجند.

- **انواع توابع تناسب:** این توابع می‌توانند به شکل اسکریپت‌های خودکار، آزمون‌های یکپارچه در خط ساخت (CI pipeline)، مانیتورینگ زمان اجرا یا حتی بازبینی‌های دستی دوره‌ای باشند. مهم این است که عینی و کمی باشند ⁴⁴ . مثلاً "خوانایی کد" یک معیار سلیقه‌ای است و برای تابع تناسب مناسب نیست، اما "تعداد سیکل‌های موجود در گراف وابستگی اجزا" یا "زمان پاسخ سرویس X تحت بار Y" کمی و قابل سنجش است ⁴⁴ .

- **نمونه‌ها:**

- تابع تناسب ساختاری: با استفاده از ابزارهایی مثل ArchUnit (برای جاوا) می‌توانید قانونی تنظیم کنید که **هیچ وابستگی چرخه‌ای** بین پکیج‌ها یا ماژول‌های سطح بالا وجود نداشته باشد ⁴⁵ . این قانون کد را تحلیل می‌کند و اگر مثلاً مؤلفه A به B وابسته باشد و B هم به A، تست شکست می‌خورد. بدین ترتیب، از تشکیل حلقه‌های ناخواسته جلوگیری می‌کنید که می‌تواند معماری را شکننده کند.

- تابع تناسب لایه‌ای: فرض کنید معماری لایه‌ای کلاسیک دارید و قاعده این است که لایه نمایش به‌طور مستقیم به لایه داده دسترسی نداشته باشد. می‌توانید تابعی تعریف کنید که کد را بررسی کند و اگر کلاسی در لایه UI به کلاسی در لایه DataAccess وابسته بود، build را fail کند ⁴⁶ . به این ترتیب **تخلف از معماری** بلافاصله آشکار می‌شود. (مثال عملی به زبان جاوا با ArchUnit: عدم اجازه به کلاس‌های بسته `..frontend..` برای وابستگی به `..backend..` در کد) ⁴⁶ .

- تابع تناسب کارایی/زیرساخت: برای ویژگی عملکرد (Performance) می‌توانید تست‌های باری (Load Tests) خودکاری تنظیم کنید که مثلاً هر استقرار جدید را تحت تعداد مشخصی درخواست در ثانیه قرار دهند و بررسی کنند که زمان پاسخ از آستانه‌ی تعیین‌شده فراتر نرود. اگر رفت، یعنی معماری یا پیاده‌سازی جدید، تناسب عملکردی را نقض کرده است و باید بهبود یابد.

توابع تناسب را متناسب با مهم‌ترین ویژگی‌های معماری خود طراحی کنید. اگر **قابلیت تغییرپذیری** برای شما اهمیت دارد، یک تابع می‌تواند پیچیدگی یا تعداد ماژول‌های درگیر در یک تغییر کد معین را اندازه‌گیری کند. اگر **امنیت** اولویت

است، می‌توانید اسکریپتی داشته باشید که پیکربندی‌های امنیتی (مانند هدرهای امنیتی، مکانیزم‌های احراز هویت) را در خروجی build یا تنظیمات سرور بررسی کند. هدف این است که معماری شما **دارای یک "سیستم ایمنی" خودکار** باشد که انحرافات را سریع گزارش یا مهار کند ⁴⁰ ⁴⁷. این کار همچنین همکاری بین معماران و توسعه‌دهندگان را تقویت می‌کند، چون معیاری عینی برای بحث و تصمیم‌گیری فراهم می‌شود ⁴³ ⁴⁸.

۳. تحلیل و مدیریت ریسک‌های معماری

در کنار طراحی فنی، باید ریسک‌هایی را که می‌توانند موفقیت پروژه یا پایداری سیستم را تهدید کنند شناسایی و مدیریت کنید. معماری عالی بدون مدیریت ریسک، در عمل ممکن است شکست بخورد. گام‌های زیر را در نظر بگیرید:

- **شناسایی ریسک‌های معماری:** ریسک معماری می‌تواند هر چیزی باشد که مانع برآورده شدن نیازهای کیفی یا عملکردی سیستم شود. برای مثال، عدم آشنایی تیم با یک فناوری جدید یک ریسک است (زیرا ممکن است در استفاده از آن به مشکل بخورید)، یا وابستگی بیش از حد یک سرویس به شبکه ریسک است (چون شبکه ذاتاً ناپایدار است). تکنیک مؤثر برای این کار **Risk Storming** است ⁴⁹. طوفان ریسک یک کارگاه مشارکتی تیمی است که در آن افراد ابتدا به شکل مستقل ریسک‌های بالقوه را در حوزه‌های مختلف (مانند عملکرد، امنیت، افراد/تیم، فرایند، فناوری و غیره) لیست می‌کنند و سپس در قالب گروهی، این ریسک‌ها به اشتراک گذاشته شده و درباره میزان اهمیتشان اجماع حاصل می‌شود ⁴⁹ ⁵⁰. مزیت این روش آن است که دیدگاه‌های متنوعی جمع می‌شود و ریسک‌های نهفته‌ای که شاید از چشم یک نفر پنهان بماند، آشکار می‌شود.

- **ارزیابی و اولویت‌بندی ریسک‌ها:** برای هر ریسک شناسایی‌شده دو بعد را ارزیابی کنید: **احتمال وقوع ریسک** و **شدت تأثیر آن در صورت وقوع** ⁵¹. یک روش مفید **ماتریس ریسک** است که معمولاً این دو بعد را در مقیاس مثلاً ۱ تا ۹ درجه‌بندی می‌کند ⁵². با ضرب این دو عدد یا قرار دادن آن‌ها در محورهای ماتریس، می‌توانید ریسک‌ها را در دسته‌های **کم، متوسط** یا **بالا** اولویت‌بندی کنید ⁵². برای مثال ریسکی با احتمال ۸ و تأثیر ۷، امتیاز ۵۶ از ۸۱ می‌گیرد که بسیار بالا است. تمرکز خود را ابتدا روی ریسک‌های با اولویت بالا بگذارید. همچنین جهت تغییرات ریسک در گذر زمان را پیگیری کنید (مثلاً با رنگ یا فلش نشان دهید که ریسک الف در حال کاهش است یا افزایش).
- **ریسک‌های رایج و اقدامات کاهشی:** به‌طور دوره‌ای فهرست ریسک‌ها را بازبینی کنید، به‌ویژه در نقاط عطف پروژه. مواردی مانند تکنولوژی‌های ناشناخته یا تازه‌وارد را همواره ریسک بالا در نظر بگیرید ⁵³. زیرا عدم قطعیت زیادی در مورد آن‌ها وجود دارد (حتی اگر عالی به نظر برسند). برای هر ریسک برنامه‌ای جهت کاهش (Mitigation) داشته باشید. مثلاً ریسک "آشنا نبودن تیم با Kubernetes" را می‌توان با **آموزش و اجرای یک پروژه آزمایشی (PoC)** کاهش داد؛ یا ریسک "مقیاس‌پذیر نبودن ماژول X" را با **انجام تست بار پیش از تولید** مدیریت کرد. هر ریسک مسئول پیگیری داشته باشد و در هر جلسه معماری یا اسکرام به‌روزرسانی وضعیت آن بررسی شود.

توجه: اگر چند ریسک خیلی بزرگ (امتیاز بالا) دارید، شاید لازم باشد در خود معماری تجدیدنظر کنید. مثلاً اگر می‌بینید استفاده از یک چارچوب خاص یا رویکردی (مثل میکروسرویس) ریسک‌های متعددی ایجاد کرده (نیاز به افراد خیلی مجرب، پیچیدگی زیرساخت، ناامن بودن ارتباطات)، گاهی بهتر است ساده‌سازی کنید. معماری باید خدمتگذار اهداف پروژه باشد، نه معکوس.

۴. ترسیم دیاگرام‌ها و ارائه معماری

مستندسازی معماری از طریق **دیاگرام‌ها** یک راه کلیدی برای ایجاد درک مشترک بین اعضای تیم و ذی‌نفعان است ⁵⁴. در این گام:

- **استفاده از استانداردهای شناخته‌شده:** پیشنهاد می‌شود از زبان‌ها یا روش‌های مدلسازی معماری قابل فهم و رایج استفاده کنید تا سوءتفاهم پیش نیاید. یکی از رویکردهای محبوب، مدل **C4** است ⁵⁵. مدل C4 چهار سطح نما برای سیستم تعریف می‌کند:
- **Context (بستر سیستم):** یک دید بسیار کلی از سیستم و محدوده مرزهایش نسبت به کاربران و سیستم‌های خارجی.
- **Container (کانتینرها/اجزای اصلی):** نمایش مؤلفه‌های اجرای‌پذیر اصلی سیستم (مانند اپلیکیشن وب، پایگاه‌داده، سرویس‌ها) و تعاملات بین آن‌ها.

- **Component (اجزای داخلی هر کانتینر):** نمایش ساختار داخلی هر کانتینر اصلی به صورت مؤلفه‌های فرعی و چگونگی تعاملشان.
- **Code (جزئیات کد یا کلاس):** در صورت نیاز، نمایش سطح کد (مثل کلاس‌ها یا ماژول‌های کتابخانه‌ای) برای تشریح دقیق پیاده‌سازی یک جزء.
- این چهار "C" کمک می‌کنند معماری را در سطوح مختلف تجرید مستند کنید⁵⁵. از علائم و نشانه‌گذاری‌های یکسان و واضح در کل دیاگرام‌ها بهره ببرید تا مخاطب با یک زبان واحد روبرو باشد (مثلاً رنگ مشخص برای سرویس‌ها، یا استفاده از خطوط چین‌دار برای ارتباطات غیرهمزمان و پیکان توپر برای ارتباط همزمان).
- **حفظ انسجام در نماهای مختلف:** اگر بخش خاصی از سیستم را در یک دیاگرام جزئی‌تر نمایش می‌دهید، **همواره جایگاه آن بخش در تصویر بزرگ‌تر را مشخص کنید**. به این کار یکنواختی بازنمایی گفته می‌شود. برای مثال، اگر یک اسلاید بزرگ از کل معماری دارید و بعد روی سرویس "پرداخت" زوم می‌کنید و اجزای داخلی آن را در اسلاید بعدی نشان می‌دهید، بهتر است در گوشه‌ای از آن اسلاید دوم، یک تصویر کوچک از کل سیستم داشته باشید و هایلایت کنید که سرویس پرداخت در کجای آن قرار دارد. این کار مانع گم‌شدن مخاطب در جزئیات شده و ارتباط بین سطوح مختلف دیاگرام‌ها را واضح نگه می‌دارد.
- **اجتناب از پیچیدگی بصری اضافه:** هر دیاگرام باید یک داستان مشخص بگوید. از شلوغ کردن بیش از حد یک نمودار با جزئیات غیرضروری خودداری کنید. بهتر است چند نمودار ساده هر کدام روی جنبه‌ای متمرکز باشند تا یک "نقشه واحد بسیار پیچیده" که همه چیز را یکجا نشان دهد. همچنین در ارائه‌ها، به توصیه‌ی معماران، از «ضدالگوی» اسلاید پر از بولت» پرهیزید - ترکیب تصاویر و متن‌های کم و گویا بهتر در خاطر می‌ماند⁵⁶.

چکلیست مرحله ۳ - تصمیمات و اقدامات:

- [] برای هر تصمیم معماری مهم (ساختار کلان، انتخاب فناوری‌های کلیدی، الگوهای طراحی بحرانی، etc) یک **رکورد تصمیم معماری (ADR)** ایجاد کرده‌ام که در آن زمینه، خود تصمیم، دلایل/شواهد و پیامدها به‌طور واضح مستند شده است.
- [] جهت تضمین پایبندی به اصول معماری، چند **تابع تناسب** طراحی و پیاده‌سازی کرده‌ام (مانند تست خودکار برای عدم وجود وابستگی چرخشی، رعایت لایه‌بندی، حفظ زمان پاسخ زیر حد مشخص، و غیره) که به صورت پیوسته یا دوره‌ای اجرا می‌شوند.
- [] فهرست ریسک‌های معماری پروژه را تهیه و اولویت‌بندی کرده‌ام؛ برای هر ریسک برنامه‌ی کاهش یا پذیرش دارم و با کمک تیم در بازه‌های زمانی منظم (یا رخدادهای کلیدی) ریسک‌ها را بازبینی می‌کنم. در صورت امکان یک جلسه **Risk Storming** برگزار شده و دیدگاه‌های تیم در شناسایی ریسک‌ها دخیل شده است.
- [] مجموعه‌ای از **دیاگرام‌های معماری** به‌روز و منسجم تهیه کرده‌ام (با استفاده از استاندارد مثل C4 یا معادل آن) که منظرهای مختلف سیستم را پوشش می‌دهند. این دیاگرام‌ها در مخزن پروژه یا ویکی در دسترس هستند و برای ارائه به ذی‌نفعان غیر فنی نیز نسخه‌های ساده‌تری آماده کرده‌ام.

</hr>

مرحله چهارم: پویایی، تکامل و نگهداشت معماری (Evolution & Maintenance)

معماری نرم‌افزار یک فعالیت یک‌باره نیست، بلکه به همراه توسعه سیستم باید تکامل یابد. نقش شما به عنوان معمار در مرحله اجرا و پس از آن نیز ادامه دارد. در این مرحله روی کارهای مداومی تمرکز داریم که معماری را زنده و موثر نگه می‌دارند.

۱. حفظ تعادل بین معماری و کدنویسی (معمار Hands-on)

یک معمار موفق معمولاً تا حدی درگیر پیاده‌سازی نیز هست. این امر دو فایده مهم دارد: اولاً درک بهتری از پیاده‌سازی عملی تصمیمات معماری به‌دست می‌آورید و ثانیاً احترام و همدلی تیم توسعه را جلب می‌کنید (آن‌ها شما را جدا از

خودشان نمی‌بینند). اما نکته مهم **تعادل** است 57 :

- **حضور در کدنویسی بدون تبدیل شدن به گلوگاه:** سعی کنید در بخش‌هایی از کد مشارکت کنید که ریسک یا عدم قطعیت بالایی دارند - مانند نوشتن **Proof of Concept (PoC)** برای یک جزء جدید یا تست یک فناوری نوین 57. این کار به کاهش ریسک‌های معماری کمک می‌کند (چون پیش از پیاده‌سازی کامل، مسائل را می‌یابید). اما در عین حال مراقب باشید که مالکیت قسمت‌های حیاتی و مسیر بحرانی توسعه را کاملاً در دست نگیرید 57. اگر تمام کدنویسی‌های مهم فقط توسط شما انجام شود، در غیاب شما تیم دچار مشکل می‌شود و خود شما هم از انجام وظایف کلان معماری باز می‌مانید.

- **کدنویسی در جهت هدایت نه انجام کل کار:** بهتر است به جای کدنویسی ویژگی‌های کسب‌وکاری روزمره، تمرکز خود را بر کارهایی بگذارید که **مسیر را برای سایرین هموار می‌کند** - مثل ایجاد چارچوب‌های پایه، تنظیمات معماری، یا حل نمونه‌ای یک مشکل که بقیه بتوانند طبق الگوی شما پیش بروند. به تعبیر دیگر، به تیم در حرکت سریع‌تر کمک کنید، اما خودتان تنها کسی نباشید که کار را جلو می‌برد.

- **بازبینی و mentoring:** در عین کدنویسی، حتماً زمانی را هم به **بررسی کدهای تیم** و منتورینگ اختصاص دهید. با این کار می‌توانید انحراف احتمالی از تصمیمات معماری را زود تشخیص دهید و دانش معماری را نیز به برنامه‌نویسان منتقل کنید.

❗ **اصل مهم:** درگیر شدن در جزئیات فنی (Coding) نباید شما را از **تصویر بزرگ** غافل کند. همواره نگاهی به معماری کلان و یکپارچگی اجزا داشته باشید. اگر دیدید مشغله‌های کدنویسی شما را از پایش تصمیمات کلی باز داشته، یک قدم عقب‌تر بردارید و تعادل را اصلاح کنید.

۲. رهبری و راهبری تیم توسعه

معمار نرم‌افزار علاوه بر جنبه‌های فنی، یک **رهبر فنی** برای تیم به‌شمار می‌آید. جنبه‌های کلیدی رهبری شما عبارتند از:

- **مرزبندی منعطف (Elastic Boundaries):** بسته به عوامل مختلف، میزان دخالت مستقیم شما در تصمیمات جزئی و کار روزمره تیم می‌تواند کم یا زیاد باشد. این عوامل شامل **تجربه و آشنایی تیم با معماری**، **اندازه تیم**، **پیچیدگی پروژه** و **مدت زمان پروژه** هستند. مثلاً اگر تیمی تازه‌کار یا پروژه‌ای بسیار حساس دارید، احتمالاً باید نظارت نزدیک‌تری داشته باشید (مرز کنترل تنگ‌تر). اما اگر تیمی مجرب و خودسازمان (Self-organizing) است، می‌توانید تصمیم‌گیری‌های بیشتری را به آن‌ها تفویض کنید. هنر شما این است که سطح مناسب هدایت و آزادی را تشخیص دهید تا نه تیم را دلسرد و محدود کنید، نه اینکه کاملاً آن‌ها را رها کنید که باعث واگرایی شود.

- **ارتباط مؤثر - اصل 4C:** چهار عنصر Communication, Collaboration, Clarity, Conciseness را سرلوحه تعاملات خود قرار دهید. یعنی: ارتباط باز و صریح با اعضای تیم داشته باشید، آن‌ها را در تصمیمات دخیل کنید (همکاری)، مفاهیم پیچیده را با شفافیت و ساده‌سازی (شفافیت بیان) منتقل کنید، و در عین حال مختصر و مفید صحبت کنید (پرحرفی و ابهام را کنار بگذارید). این رویکرد ارتباطی جلوی ایجاد **پیچیدگی تصادفی** را می‌گیرد و فرهنگ سالمی در تیم ایجاد می‌کند. برای مثال، وقتی قرار است تصمیمی را ابلاغ کنید، ابتدا دلیل و چرایی آن را بگویید (شفافیت) 58، سپس در بحث و اجرای آن اجازه دهید اعضا نظر دهند (همکاری). همچنین مستندات معماری را بیش از حد طولانی و مفصل نکنید - به اندازه‌ای بنویسید که برای فهم و رعایت کافی باشد (اختصار).

- **الگوسازی و نفوذ (Influence):** به جای صدور دستور صرف، سعی کنید با الگو بودن و اقناع، فرهنگ مورد نظر را ترویج دهید. نمونه: اگر می‌خواهید کیفیت کد بالا برود، خودتان در مراجعات کد دقیق و سازنده بازخورد دهید. اگر معماری Clean Code را ارزش می‌دانید، بخش‌هایی از کد را که مطابق این اصول است هایلایت و تمجید کنید. اینگونه تیم یاد می‌گیرد که شما صرفاً کنترل‌کننده نیستید بلکه **همراه و راهنما** هستید.

۳. به‌روز نگه داشتن معماری و دانش فنی

فناوری با سرعت زیادی پیشرفت می‌کند و معماری‌ای که امروز عالی است ممکن است چند سال دیگر ناکارآمد شود. وظیفه شماست که هم **خودتان** و هم **معماری سیستم** را به‌روز نگه دارید.

- **یادگیری مستمر (قانون ۲۰ دقیقه):** همان‌طور که در مرحله اول اشاره شد، روزانه زمانی را به یادگیری اختصاص دهید 2. این یادگیری می‌تواند خواندن مقالات معماری، شرکت در وبینارها، دنبال کردن وبلاگ‌های فنی یا تمرین

ابزارهای جدید باشد. حتی می‌توانید این کار را به صورت فعالیت تیمی درآورید - مثلاً هر هفته یک جلسه کوتاه Knowledge Sharing داشته باشید که در آن یکی از اعضا درباره تکنولوژی جدیدی که بررسی کرده صحبت کند. استمرار در یادگیری باعث می‌شود در تصمیم‌گیری‌های بعدی گزینه‌های بهتری در آستین داشته باشید.

- **رادار فناوری شخصی:** ایده‌ی Tech Radar که توسط شرکت ThoughtWorks معروف شد، می‌تواند به صورت شخصی هم به‌کار رود ⁵⁹ ⁶⁰. یک رادار فناوری شخصی برای خود ایجاد کنید. فناوری‌ها، ابزارها یا رویکردهای جدیدی را که برخورد می‌کنید در چهار حلقه‌ی نگهداشت (Hold)، ارزیابی (Assess)، آزمایش (Trial) و پذیرش (Adopt) دسته‌بندی کنید ⁶⁰. برای مثال، اگر Kubernetes را شنیده‌اید اما هنوز تجربه ندارید شاید در حلقه Assess (ارزش بررسی دارد) بگذارید. یک فناوری اثبات‌شده در صنعت که قصد دارید به زودی در پروژه‌ای استفاده کنید را در Trial قرار دهید (می‌خواهید دست‌به‌کار آزمایش شوید). فناوری‌های استاندارد مثل Git یا یک چارچوب وب که کاملاً به آن اعتماد دارید در Adopt خواهد بود (جزو بایدهای شما). این رادار را هر چند وقت یکبار بازبینی کنید و موارد را جابجا کنید. چنین ابزاری به شما دید می‌دهد که روی چه چیزهایی باید متمرکز شوید و چه چیزهایی را فعلاً کنار بگذارید.

- **بازنگری معماری در طول زمان:** پس از استقرار اولیه سیستم، کار معماری تمام نمی‌شود. با هر تغییر مهم در نیازمندی‌ها یا هر مشکلی که در اجرا نمایان می‌شود، معماری را بازنگری کنید. اگر اضافه کردن یک قابلیت جدید بیش از حد پیچیده به نظر می‌رسد، شاید نشانه‌ای باشد که ساختار فعلی نیاز به تغییر (Refactoring معماری یا حتی Re-architecture) دارد. از انجام تغییرات تکاملی در معماری نهراسید - البته این کار را کنترل‌شده انجام دهید: اول تأثیر را در قالب PoC یا تحلیل مستند بسنجید، سپس در صورت اطمینان، آن را به معماری رسمی بدل کنید و ADRهای مربوطه را نیز اضافه/به‌روزرسانی کنید.

چک‌لیست مرحله ۴ - تصمیمات و اقدامات:

- [] همچنان که پروژه در حال پیاده‌سازی است، به عنوان معمار در بخش‌های کلیدی hands-on بوده‌ام: با اجرای نمونه‌های اولیه یا حل مسائل فنی دشوار، به هدایت تیم کمک کرده‌ام، بدون آن‌که خود به گلوگاه تبدیل شوم.

- [] رویکرد رهبری خود را متناسب با شرایط تنظیم کرده‌ام: به اعضای تیم اعتماد دارم و در عین حال نظارت کافی برای جلوگیری از انحراف را انجام می‌دهم. بازخوردهایم بر مبنای اصل 4C شفاف، مختصر و همراه با مشارکت تیم بوده است.

- [] برنامه‌ی یادگیری مداوم شخصی دارم و دانش فنی‌ام را به‌روز نگه می‌دارم (مطالعه مستمر، شرکت در رویدادها، پیگیری روندهای نو). همچنین یک "رادار فناوری" ایجاد کرده‌ام تا فناوری‌های نوظهور را رصد و اولویت‌بندی کنم ⁶⁰.

- [] معماری سیستم را در بازه‌های زمانی مناسب بازبینی می‌کنم. هر جا لازم باشد، با مشورت تیم تغییرات معماری را اعمال کرده و مستند می‌کنم تا سیستم در بلندمدت چابک و قابل نگهداری بماند.

</hr>

نتیجه‌گیری

با دنبال کردن این راهنمای گام‌به‌گام، شما یک نقشه راه تصمیم‌گیری برای معماری سیستم خود خواهید داشت که از مرحله تحلیل نیازمندی‌ها تا مراحل نگهداشت و تکامل را پوشش می‌دهد. هر مرحله مجموعه‌ای از پرسش‌های کلیدی را مطرح می‌کند که پاسخ به آن‌ها جهت معماری را روشن‌تر می‌سازد. به خاطر داشته باشید که معماری یک فرآیند ایستا نیست؛ بلکه پویا و در تعامل با توسعه سیستم است. بنابراین همواره آماده باشید تا با تغییر شرایط، تصمیمات خود را بازبینی کنید. یک معمار موفق، ترکیبی از دانش فنی به‌روز، درک عمیق کسب‌وکار و مهارت ارتباطی و رهبری را به‌کار می‌گیرد تا بهترین راه‌حل ممکن را - در بستر محدودیت‌ها و نیازمندی‌های موجود - ارائه دهد.

با مستندسازی تصمیمات، ارزیابی مداوم کیفیت معماری (از طریق شاخص‌ها و توابع تناسب)، و همکاری نزدیک با تیم توسعه، معماری سیستم شما نه تنها روی کاغذ بلکه در عمل نیز محقق خواهد شد. این راهنمای تصمیم‌گیری می‌تواند به عنوان چک‌لیست همراه شما باشد تا هیچ بعد مهمی را از قلم نیاندازید و در هر گام بدانید روی چه چیزی باید تمرکز کنید. اکنون می‌توانید با اطمینان بیشتری سفر معماری خود را آغاز کنید، چرا که نقشه راه روشنی پیش‌روی شماست. موفق باشید!

Book notes: Fundamentals 57 25 24 23 22 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1
of Software Architecture
[/https://danlebrero.com/2021/11/17/fundamentals-of-software-architecture-summary](https://danlebrero.com/2021/11/17/fundamentals-of-software-architecture-summary)

Should you use microservices? | Rehan van der Merwe 58 28 27 26 21 20 19 18
<https://rehanvdm.com/blog/should-you-use-microservices>

Orchestration vs. Choreography in Microservices - GeeksforGeeks 32 29
[/https://www.geeksforgeeks.org/system-design/orchestration-vs-choreography](https://www.geeksforgeeks.org/system-design/orchestration-vs-choreography)

Distributed workflow in microservices (Orchestration vs Choreography) 30
<https://harish-bhattbhatt.medium.com/distributed-workflow-in-microservices-orchestration-vs-choreography-cf03cfef25db>

Orchestration vs. Choreography in Microservices - ByteByteGo 31
[/https://bytebytego.com/guides/orchestration-vs-choreography-microservices](https://bytebytego.com/guides/orchestration-vs-choreography-microservices)

Fundamentals of Software Architecture Book - Part 5 | codesandtags 56 55 54 53 51 33
[/https://www.codesandtags.io/blog/posts/fundamentals-of-software-architecture-part-5](https://www.codesandtags.io/blog/posts/fundamentals-of-software-architecture-part-5)

ADR process - AWS Prescriptive Guidance 39 38 37 36 35 34
<https://docs.aws.amazon.com/prescriptive-guidance/latest/architectural-decision-records/adr-process.html>

Fitness Functions for Your Architecture - InfoQ 48 47 46 45 44 43 42 41 40
[/https://www.infoq.com/articles/fitness-functions-architecture](https://www.infoq.com/articles/fitness-functions-architecture)

Risk storming 52 50 49
<https://icepanel.io/blog/2023-06-15-risk-storming>

Build Your Own Technology Radar | Thoughtworks United States 60 59
<https://www.thoughtworks.com/en-us/insights/blog/build-your-own-technology-radar>